

SPACE ORBITERS

Practical Exam Project Report

Dockerized 3-Tier Application • PostgreSQL • Docker Compose • AWS EC2

Structured implementation evidence with additional production deployment enhancements

Repository	github.com/shashankcodes-10/SPACE_ORBITERS
Deployment	space.shashankpipal.in
Database	PostgreSQL 16
Deployment Platform	AWS EC2 (Ubuntu)

Prepared as a structured evidence report

Report Overview

This report presents the implementation in a step-by-step format. The main sections document the practical project work, while DNS, SSL/HTTPS, Nginx reverse proxying, and Jenkins are presented separately as additional production-oriented enhancements.

Core implementation	Project structure, Git practices, architecture, Dockerization, image optimization, Compose, networking, persistence, and EC2 deployment.
Additional enhancements	Elastic IP, GoDaddy DNS, Nginx reverse proxy, Let's Encrypt HTTPS, and Jenkins CI/CD/DevSecOps setup.
Evidence format	Screenshots are kept in the supplied sequence except where specific screenshots were intentionally moved to the topic they prove.

STEP 1

Project Foundation & Git Practices

The project was organized as a full-stack space exploration application and maintained through Git/GitHub with incremental, meaningful changes.

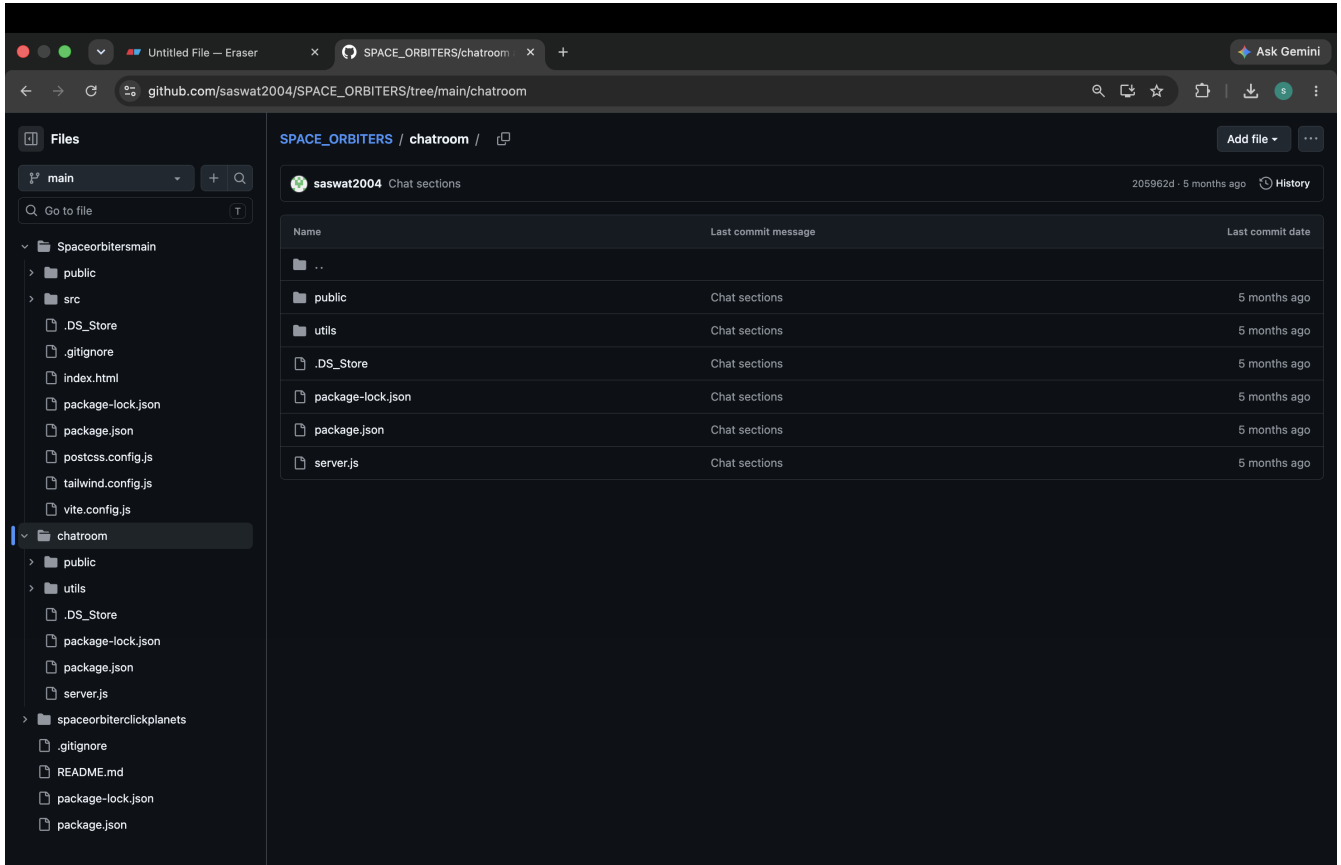


Figure 1 — GitHub repository and project contents.

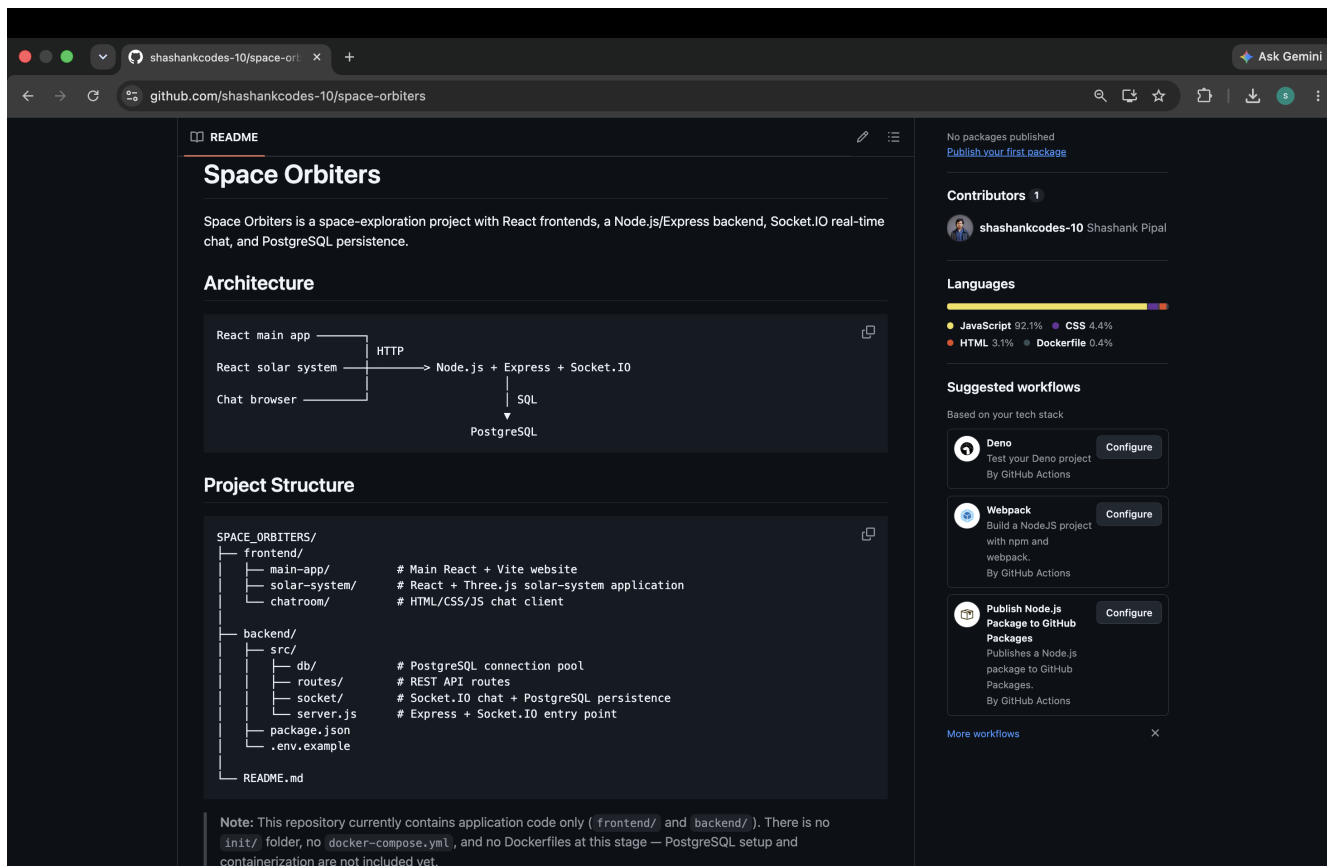


Figure 2 — Repository README and project structure.

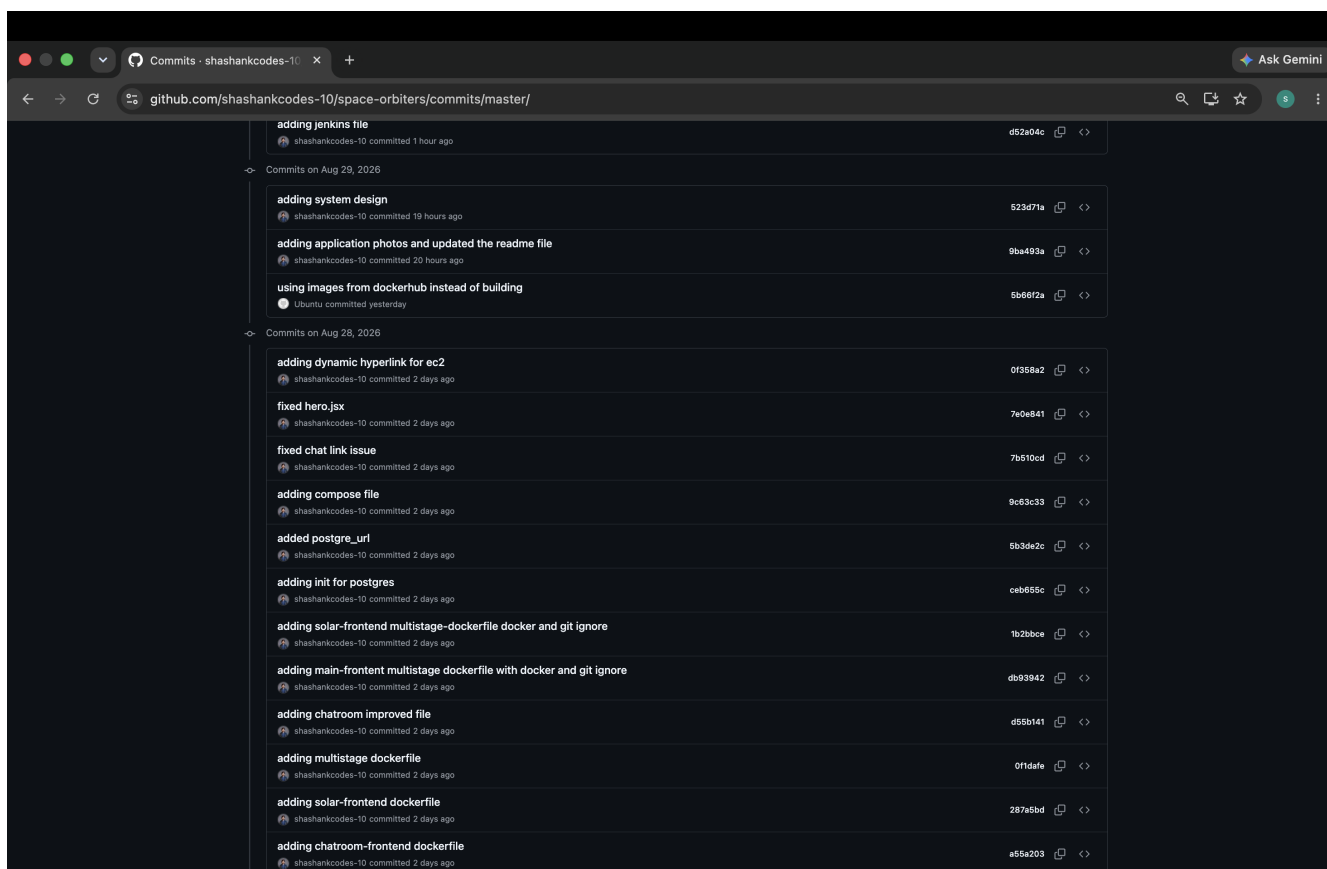


Figure 3 — Git commit history showing incremental project changes, fixes, Docker work, PostgreSQL work, and deployment-related commits.

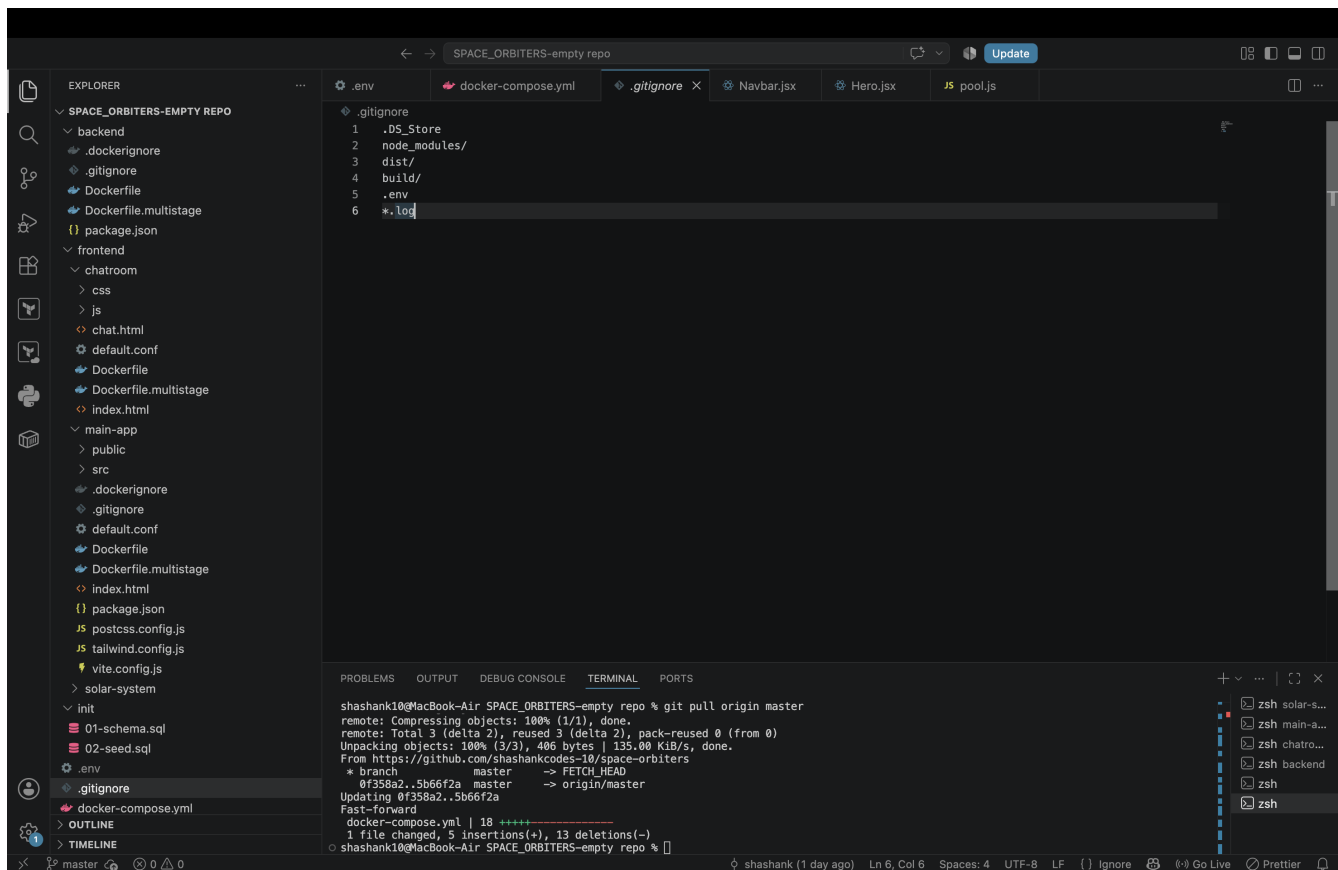


Figure 4 — .gitignore and repository structure in the development workspace.

STEP 2

System Design & Architecture

The architecture uses a 3-tier application model with presentation, application, and data responsibilities separated across containers.

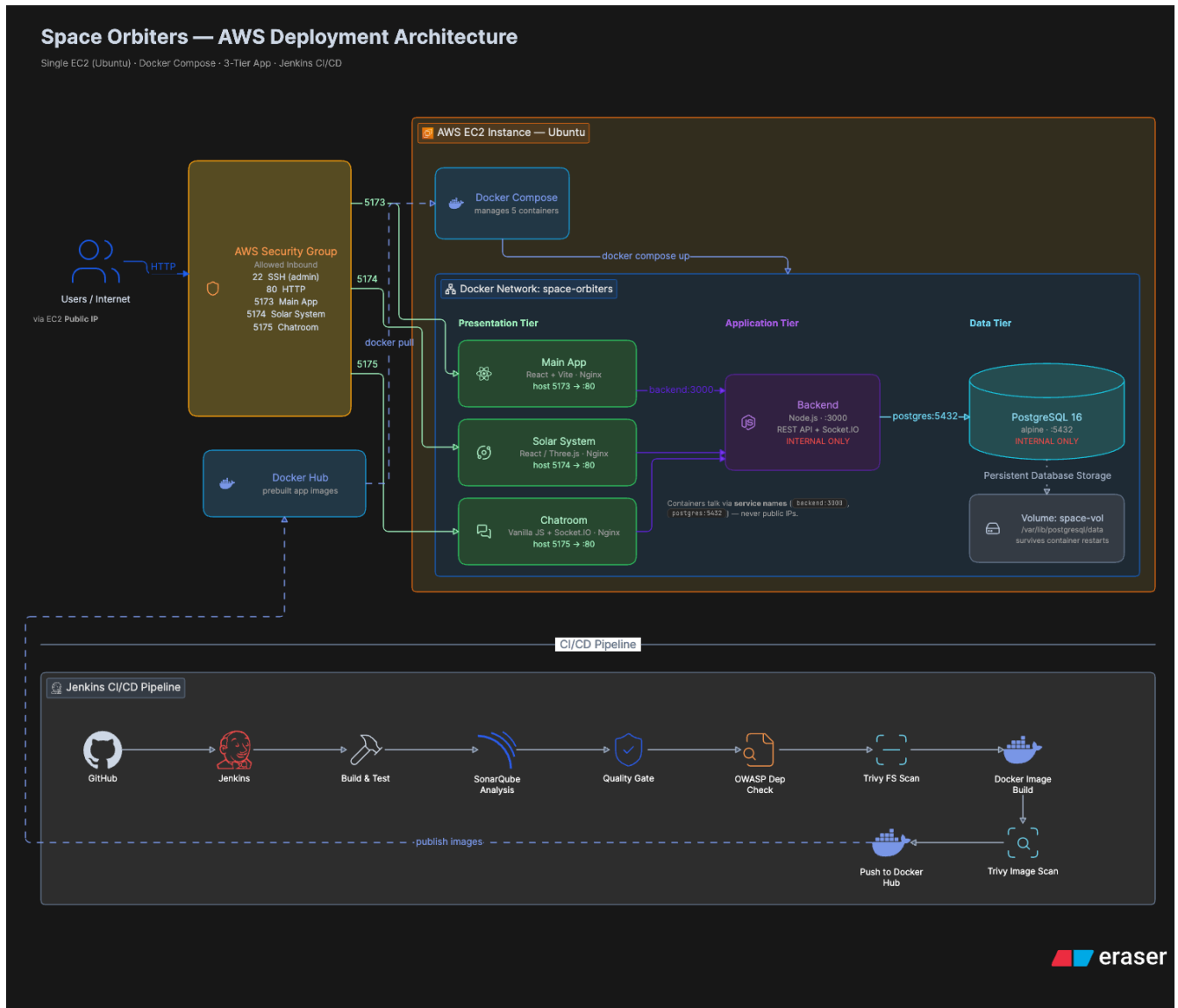


Figure 5 — Space Orbiters AWS deployment architecture and CI/CD design.

STEP 3

Dockerization

Each application component was containerized. Multi-stage Dockerfiles were introduced for the backend and frontend applications to produce leaner runtime images.

```
frontend > solar-system > 🚢 Dockerfile > ...  
1  #Dockerfile for solar-system  
2  FROM node:21  
3  
4  WORKDIR /app  
5  
6  COPY . .  
7  
8  RUN npm install  
9  
10 EXPOSE 5174  
11  
12 CMD ["npm", "run", "dev"]
```

Figure 6 — Solar System frontend Dockerfile.

```
frontend > main-app >  Dockerfile > ...  
1   #Dockerfile for main-app  
2   FROM node:21  
3  
4   WORKDIR /app  
5  
6   COPY . .  
7  
8   RUN npm install  
9  
10  EXPOSE 5173  
11  
12  CMD ["npm", "run", "dev"]
```

Figure 7 — Main App Dockerfile.

```
frontend > chatroom >  Dockerfile > ...  
1   #Dockerfile for chatroom  
2   FROM nginx:latest  
3  
4   WORKDIR /app  
5  
6   COPY . /usr/share/nginx/html  
7  
8   COPY default.conf /etc/nginx/conf.d/default.conf  
9  
10  EXPOSE 80  
11  
12  CMD ["nginx", "-g", "daemon off;"]
```

Figure 8 — Chatroom Dockerfile.

```

backend > 🚢 Dockerfile > ...
1  #Dockerfile for backend
2  FROM node:21
3
4  WORKDIR /app
5
6  COPY . .
7
8  RUN npm install
9
10 EXPOSE 3000
11
12 CMD ["npm","start"]

```

Figure 9 — Backend Dockerfile.

```

frontend > solar-system > 🚢 Dockerfile.multistage > ...
1  #-----builder stage-----
2  FROM dhi.io/node:24-alpine3.23-dev AS builder
3
4  WORKDIR /app
5
6  COPY package.*json ./
7
8  RUN npm install
9
10 COPY . .
11
12 RUN npm run build
13 #-----runner stage-----
14 FROM dhi.io/nginx:1-alpine3.23
15
16 COPY --from=builder /app/dist /usr/share/nginx/html
17 COPY default.conf /etc/nginx/conf.d/default.conf
18
19 EXPOSE 80

```

Figure 10 — Solar System multi-stage Dockerfile.

```

frontend > main-app > Dockerfile.multistage > ...
1  #-----builder stage-----
2  FROM dhi.io/node:24-alpine3.23-dev AS builder
3
4  WORKDIR /app
5
6  COPY package.*json ./
7
8  RUN npm install
9
10 COPY . .
11
12 RUN npm run build
13 #-----runner stage-----
14 FROM dhi.io/nginx:1-alpine3.23
15
16 COPY --from=builder /app/dist /usr/share/nginx/html
17 COPY default.conf /etc/nginx/conf.d/default.conf
18
19 EXPOSE 80

```

Figure 11 — Main App multi-stage Dockerfile.

```

frontend > chatroom > Dockerfile.multistage > ...
1  #improved dockerfile
2  FROM dhi.io/nginx:1-alpine3.23
3
4  WORKDIR /app
5
6  COPY . /usr/share/nginx/html
7
8  COPY default.conf /etc/nginx/conf.d/default.conf
9
10 EXPOSE 80
11

```

Figure 12 — Chatroom multi-stage Dockerfile.

```
backend > 🐳 Dockerfile.multistage > ...
1  #-----builder stage-----
2  FROM dhi.io/node:24-alpine3.23-dev AS builder
3
4  WORKDIR /app
5
6  COPY package.*json ./
7
8  RUN npm install --omit=dev
9
10 #-----runner stage-----
11 FROM dhi.io/node:24-alpine3.23
12
13 WORKDIR /app
14
15 ENV NODE_ENV=production
16
17 COPY --from=builder /app/node_modules ./node_modules
18 COPY package.json ./
19 COPY src ./src
20
21 EXPOSE 3000
22
23 CMD ["node", "src/server.js"]
```

Figure 13 — Backend multi-stage Dockerfile.

STEP 4

Docker Image Optimization

Multi-stage builds reduced the final runtime image sizes by separating build dependencies from production runtime layers.

☐		front-solar	latest	7773e9c09be5	30 minutes ago	2.75 GB				
☐		front-main	latest	582eaf708227	25 minutes ago	2.3 GB				
☐		front-chat	latest	4ee9d78a1a93	12 minutes ago	257.21 MB				
☐		backend	latest	c4363aae3024	2 minutes ago	1.61 GB				

Figure 14 — Docker image sizes after optimization/building.

☐		front-solar	multistage	b4e2ccab68fc	6 minutes ago	33.22 MB				
☐		front-main	multistage	da70ac650b28	3 minutes ago	202.4 MB				
☐		front-chat	new	60af62805960	2 minutes ago	18.02 MB				
☐		backend	multistage	c18a448bc713	44 seconds ago	164.76 MB				

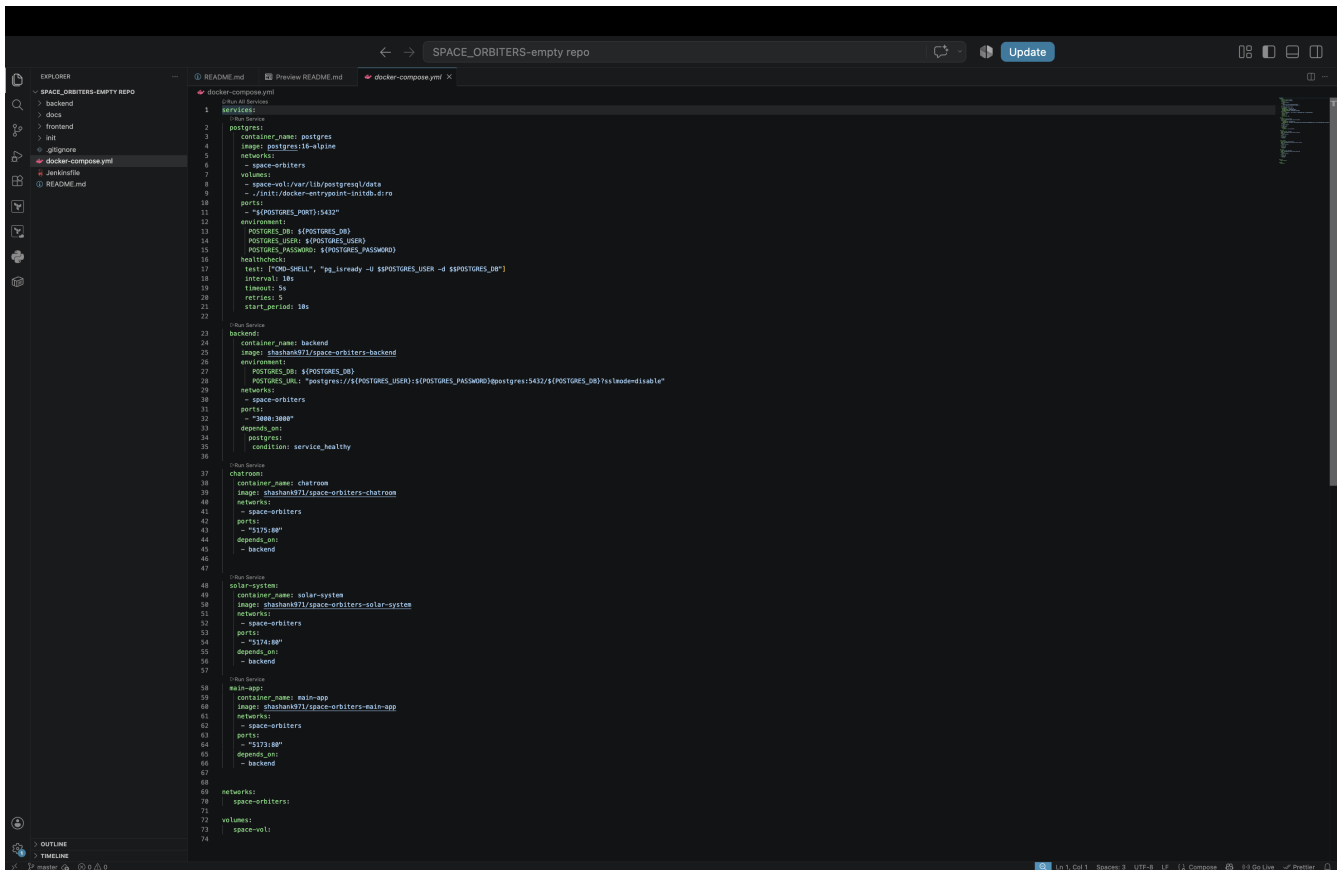
Figure 15 — Docker image size comparison evidence.

Observed optimized sizes: front-solar $\approx$ 33 MB • front-main $\approx$ 202 MB • front-chat $\approx$ 18 MB • backend $\approx$ 165 MB.

STEP 5

Docker Compose & Networking

Docker Compose coordinates the application services and connects them through the project Docker network. The Compose file also defines database health checking, dependencies, volumes, and service configuration.



```
1 services:
2   postgres:
3     container_name: postgres
4     image: postgres:16-alpine
5     networks:
6       - space-orbiters
7     volumes:
8       - spacevol:/var/lib/postgresql/data
9       - ./init:/docker-entrypoint-initdb.d:ro
10    ports:
11      - "${POSTGRES_PORT}:5432"
12    environment:
13      POSTGRES_DB: ${POSTGRES_DB}
14      POSTGRES_USER: ${POSTGRES_USER}
15      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
16    healthcheck:
17      test: ["CMD-SHELL", "pg_isready -U $POSTGRES_USER -d $POSTGRES_DB"]
18      interval: 10s
19      timeout: 5s
20      retries: 5
21      start_period: 10s
22
23   backend:
24     container_name: backend
25     image: shashan971/space-orbiters-backend
26     environment:
27       POSTGRES_DB: ${POSTGRES_DB}
28       POSTGRES_USER: ${POSTGRES_USER}
29       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
30     networks:
31       - space-orbiters
32     ports:
33       - "3000:3000"
34     depends_on:
35       postgres:
36         condition: service_healthy
37
38   chatroom:
39     container_name: chatroom
40     image: shashan971/space-orbiters-chatroom
41     networks:
42       - space-orbiters
43     ports:
44       - "5175:80"
45     depends_on:
46       - backend
47
48   solar-system:
49     container_name: solar-system
50     image: shashan971/space-orbiters-solar-system
51     networks:
52       - space-orbiters
53     ports:
54       - "5174:80"
55     depends_on:
56       - backend
57
58   main-app:
59     container_name: main-app
60     image: shashan971/space-orbiters-main-app
61     networks:
62       - space-orbiters
63     ports:
64       - "5173:80"
65     depends_on:
66       - backend
67
68 networks:
69   space-orbiters:
70
71 volumes:
72   space-vol:
73
```

Figure 16 — Docker Compose configuration showing services, network, PostgreSQL volume, healthcheck, and dependencies.

```
ubuntu@ip-172-31-46-234:~$ ls
space-orbiters
ubuntu@ip-172-31-46-234:~$ cd space-orbiters/
ubuntu@ip-172-31-46-234:~/space-orbiters$ ls
README.md backend docker-compose.yml frontend init
ubuntu@ip-172-31-46-234:~/space-orbiters$ docker compose up -d
[+] Running 6/6
  ✓ Network space-orbiters_space-orbiters    Created                                0.1s
  ✓ Container postgres                        Healthy                                6.1s
  ✓ Container backend                        Started                                6.2s
  ✓ Container main-app                       Started                                6.8s
  ✓ Container chatroom                      Started                                6.6s
  ✓ Container solar-system                  Started                                6.8s
ubuntu@ip-172-31-46-234:~/space-orbiters$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
bbe5a32117bc  shashank971/space-orbiters-solar-system  "nginx -g 'daemon of..." 12 seconds ago Up 6 seconds  8080/tcp, 0.0.0.0:5174->80/tcp, [::]:5174->80/tcp  solar-system
a84979f4c1df  shashank971/space-orbiters-chatroom      "nginx -g 'daemon of..." 12 seconds ago Up 6 seconds  8080/tcp, 0.0.0.0:5175->80/tcp, [::]:5175->80/tcp  chatroom
cf5942677fd2  shashank971/space-orbiters-main-app      "nginx -g 'daemon of..." 12 seconds ago Up 6 seconds  8080/tcp, 0.0.0.0:5173->80/tcp, [::]:5173->80/tcp  main-app
47749cef305b  shashank971/space-orbiters-backend      "node src/server.js"      12 seconds ago Up 6 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  backend
b94f795f4766  postgres:16-alpine                    "docker-entrypoint.s..." 12 seconds ago Up 12 seconds (healthy)  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp  postgres
```

Figure 17 — Compose startup and running-container verification on the EC2 environment.

STEP 6

Persistent Database Data

PostgreSQL is used as the persistent data tier. Database initialization is supplied through SQL scripts and PostgreSQL data is stored in the Docker volume used by the Compose deployment.

Persistence evidence: The supplied demonstration video contains the persistence verification, showing the database/data workflow used for the project.

[Open the Google Drive persistence-proof video](#)

The video link is intentionally included inside the persistence section as evidence for this requirement; no separate project-demo section is added.

STEP 7

AWS EC2 Deployment

The containerized application was deployed on an AWS EC2 Ubuntu instance. The security-group evidence documents the network access configured for the deployed environment.

The screenshot shows the AWS Management Console interface for an EC2 instance. The left sidebar contains navigation links for EC2, Instances, Images, Elastic Block Store, and Network & Security. The main content area displays the 'Security groups' page for the instance 'i-0c053d85514130738'. The 'Inbound rules' table lists five rules, each with a 'Security group rule ID', 'Port range', 'Protocol', 'Source', and a 'launch-wizard' link. The 'Outbound rules' table lists one rule with a 'Security group rule ID', 'Port range', 'Protocol', 'Destination', and a 'launch-wizard' link.

Name	Security group rule ID	Port range	Protocol	Source	Security
-	sgr-055a33b76b5e0887a	5175	TCP	0.0.0.0/0	launch-w
-	sgr-0846c49f51d429694	5174	TCP	0.0.0.0/0	launch-w
-	sgr-0067844d004f05b7a	22	TCP	0.0.0.0/0	launch-w
-	sgr-0b3ae5a83fe13d93d	80	TCP	0.0.0.0/0	launch-w
-	sgr-0eb54c703a90b8142	5173	TCP	0.0.0.0/0	launch-w

Name	Security group rule ID	Port range	Protocol	Destination	Security
-	sgr-00aa20b7a6934b7d3	All	All	0.0.0.0/0	launch-w

Figure 18 — EC2 security-group inbound rules used during deployment.

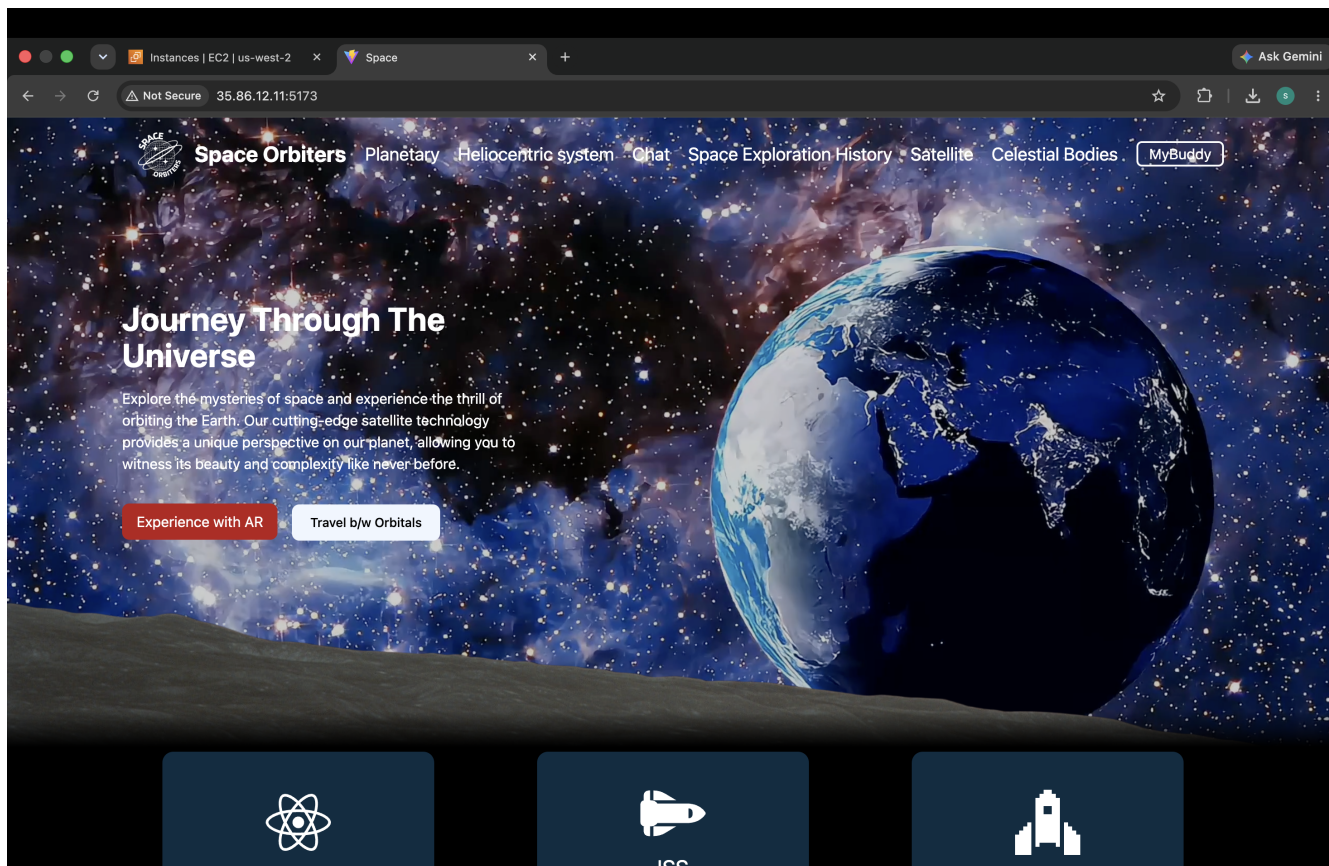


Figure 19 — Main Space Orbits application running.

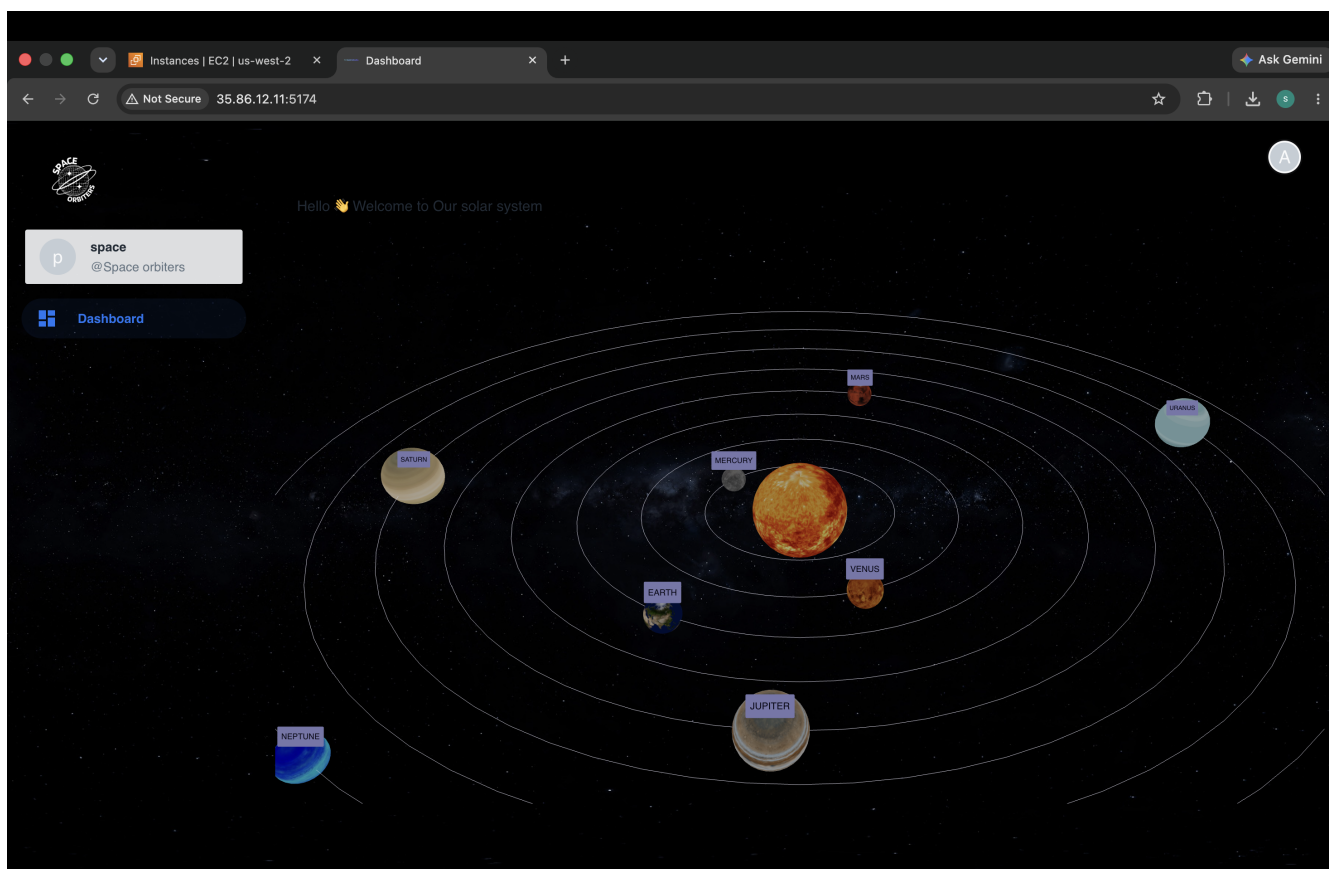


Figure 20 — Solar System application running.

STEP A

Additional Enhancement — Elastic IP & DNS

A stable public address was associated with the EC2 instance, and the GoDaddy DNS record was configured so the application can be reached through a named subdomain instead of an EC2 IP address.

The screenshot displays the AWS Management Console interface for an Elastic IP address. The browser address bar shows the URL: `us-west-2.console.aws.amazon.com/ec2/home?region=us-west-2#ElasticIpDetails:AllocationId=eipalloc-0aa03d9d953e40b97`. The console header includes the AWS logo, a search bar, and the region set to "United States (Oregon)". The left-hand navigation menu is expanded to "Network & Security", with "Elastic IP addresses" selected. The main content area shows the details for the Elastic IP address **35.86.12.11**. At the top right of this section are buttons for "Actions" and "Associate Elastic IP address".

Summary			
Allocated IPv4 address 35.86.12.11	Type Public IP	Allocation ID eipalloc-0aa03d9d953e40b97	Reverse DNS record -
Association ID eipassoc-06ff784a9f09f7a34	Scope VPC	Associated instance ID i-0c053d85514130738	Private IP address 172.31.46.234
Network interface ID eni-05003cd610afb845	Network interface owner account ID 593489477004	Public DNS ec2-35-86-12-11.us-west-2.compute.amazonaws.com	NAT Gateway ID -
Address pool Amazon	Network border group us-west-2	Service managed -	

Below the summary table, there is a "Tags(1)" section with a "Manage tags" button. It contains a single tag with the key "Name" and the value "space-orbiters-app".

The footer of the console shows links for "CloudShell", "Agent Toolkit for AWS", "Feedback", and "Console Mobile App", along with copyright information: "© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences".

Figure 21 — AWS Elastic IP 35.86.12.11 associated with the EC2 instance.

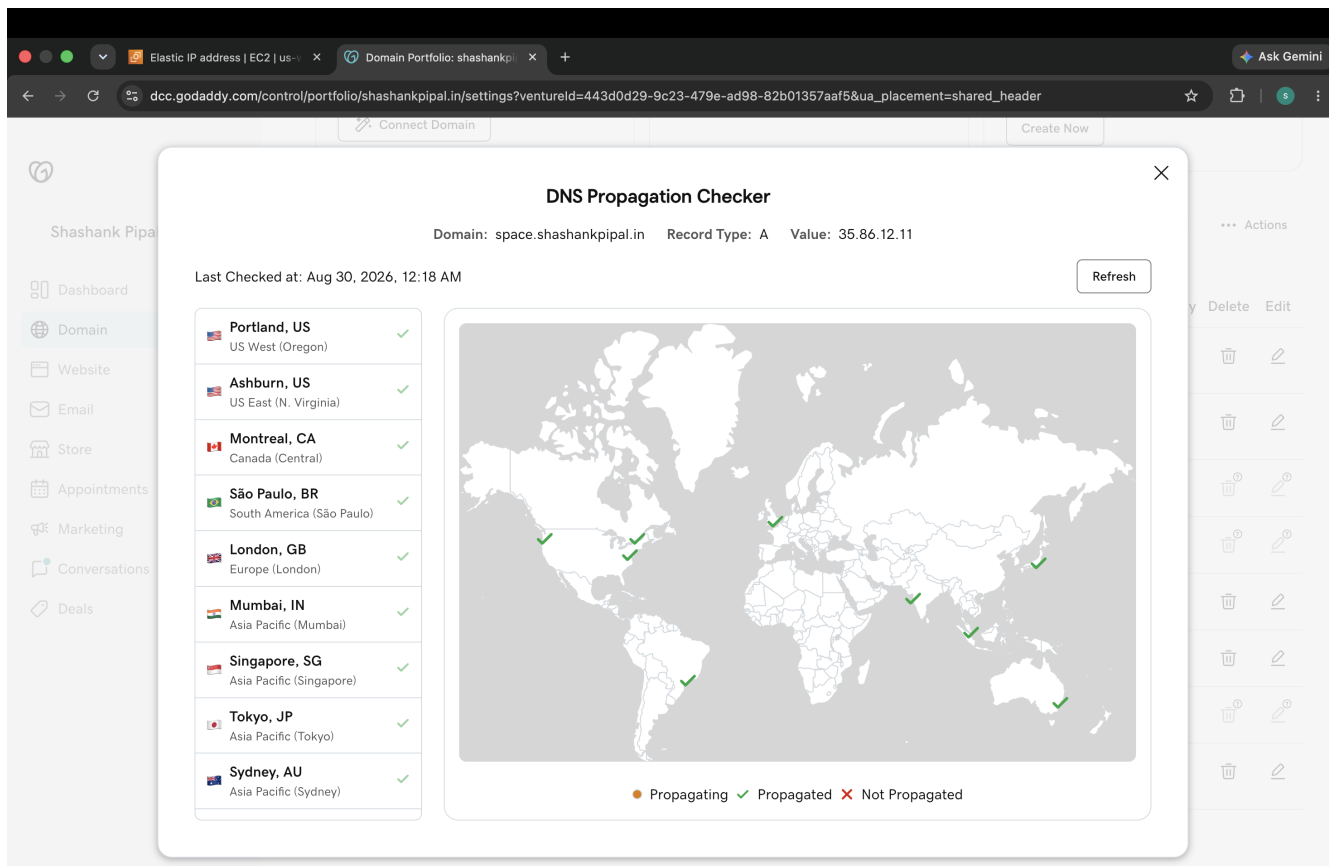


Figure 22 — GoDaddy DNS propagation check for `space.shashankpipal.in` pointing to `35.86.12.11`.

STEP B

Additional Enhancement — Nginx Reverse Proxy & HTTPS

Nginx was introduced as the public reverse proxy. It handles HTTPS and routes the application by path, while Certbot/Let's Encrypt provides the TLS certificate.

```
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ mkdir -p certbot/www/.well-known/acme-challenge
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ echo "certbot-test" > certbot/www/.well-known/acme-challenge/test.txt
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ curl http://space.shashankpipal.in/.well-known/acme-challenge/test.txt
certbot-test
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ rm -f certbot/www/.well-known/acme-challenge/test.txt
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ docker run --rm -it \
-v "$(pwd)/certbot/www:/var/www/certbot" \
-v "$(pwd)/certbot/conf:/etc/letsencrypt" \
certbot/certbot certonly \
--webroot \
--webroot-path=/var/www/certbot \
-d space.shashankpipal.in \
--email ui0shashank@gmail.com \
--agree-tos \
--no-eff-email
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Account registered.
Requesting a certificate for space.shashankpipal.in

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/space.shashankpipal.in/fullchain.pem
Key is saved at: /etc/letsencrypt/live/space.shashankpipal.in/privkey.pem
This certificate expires on 2026-11-27.
These files will be updated when the certificate renews.

NEXT STEPS:
- The certificate will need to be renewed before it expires. Certbot can automatically renew the certificate in the background, but you may need to take steps to enable that functionality. See https://certbot.org/renewal-setup for instructions.

-----
If you like Certbot, please consider supporting our work by:
* Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
* Donating to EFF: https://eff.org/donate-le
-----

ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ cd nginx-proxy/
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2/nginx-proxy$ vim default.conf
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2/nginx-proxy$ cd ..
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ docker compose exec nginx-proxy nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ docker compose up -d nginx-proxy
[*] Running 6/6
✔ Container postgres Healthy 0.6s
✔ Container backend Running 0.0s
✔ Container chatroom Running 0.0s
✔ Container main-app Running 0.0s
✔ Container solar-system Running 0.0s
✔ Container nginx-proxy Running 0.0s
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ curl -I https://space.shashankpipal.in
curl: (7) Failed to connect to space.shashankpipal.in port 443 after 104 ms: Could not connect to server
ubuntu@ip-172-31-46-234: ~/space-orbiters-ec2$ docker ps --format "table {{.Names}}\t{{.Ports}}"
```

Figure 23 — Certbot successfully obtaining the Let's Encrypt certificate for space.shashankpipal.in.


```
# HTTP → HTTPS
server {
    listen 80;
    server_name space.shashankipal.in;

    # Keep this for Let's Encrypt renewal
    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}

# HTTPS
server {
    listen 443 ssl;
    server_name space.shashankipal.in;

    ssl_certificate /etc/letsencrypt/live/space.shashankipal.in/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/space.shashankipal.in/privkey.pem;

    # Main App
    location / {
        proxy_pass http://main-app:80/;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Solar System
    location /solar/ {
        proxy_pass http://solar-system:80/;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Chatroom
    location /chat/ {
        proxy_pass http://chatroom:80/;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Backend API
    location /api/ {
        proxy_pass http://backend:3000/api/;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Figure 24 — Nginx HTTP→HTTPS redirect, TLS configuration, and path-based reverse-proxy routes.

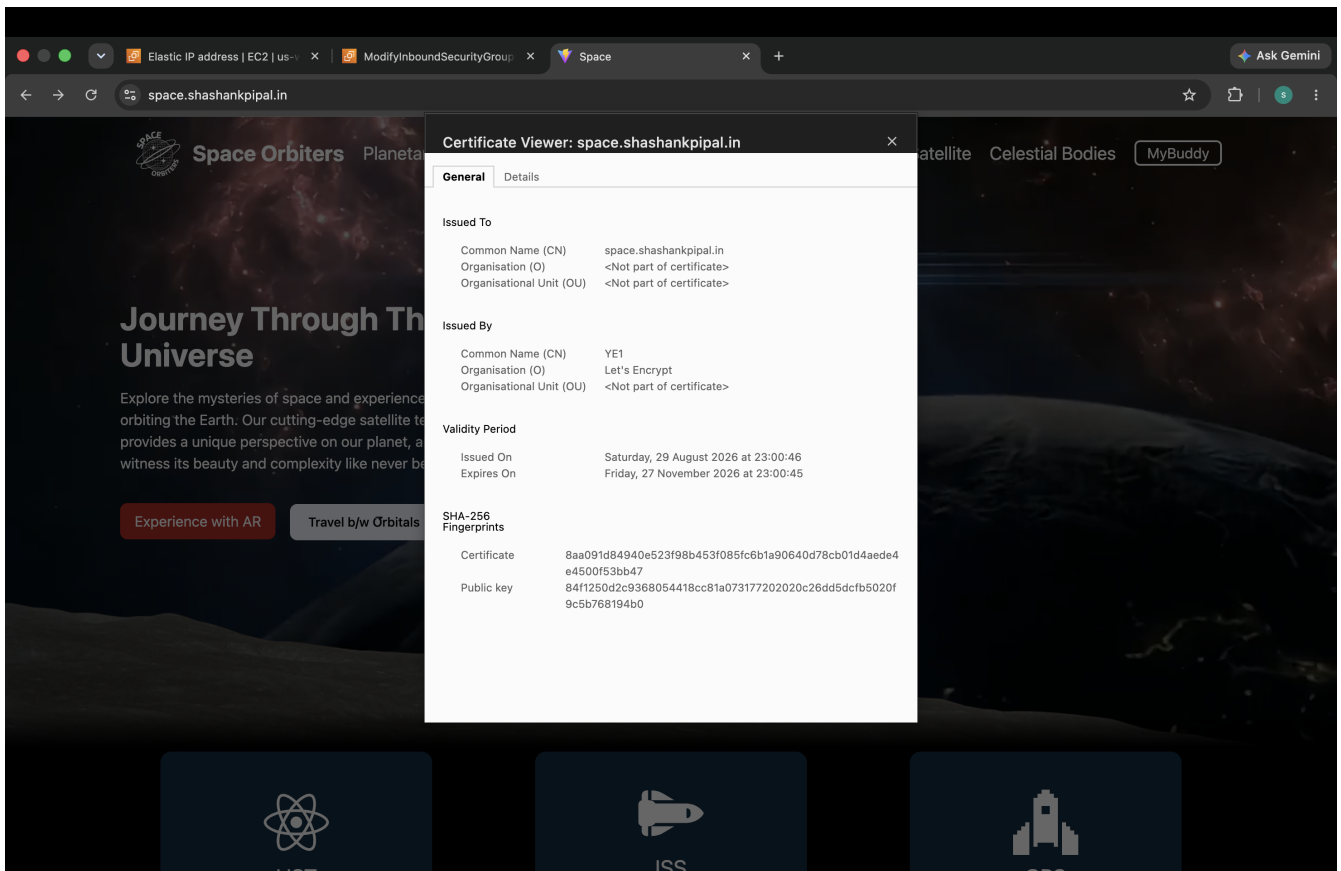


Figure 25 — Browser certificate viewer showing the Let's Encrypt certificate issued to space.shashankipal.in.

STEP C

Additional Enhancement — Jenkins CI/CD & DevSecOps

Jenkins automation was prepared using a reusable Shared Library. The pipeline is designed around source checkout, code quality, dependency/security scanning, Docker build and image scanning, Docker Hub publishing, and deployment.

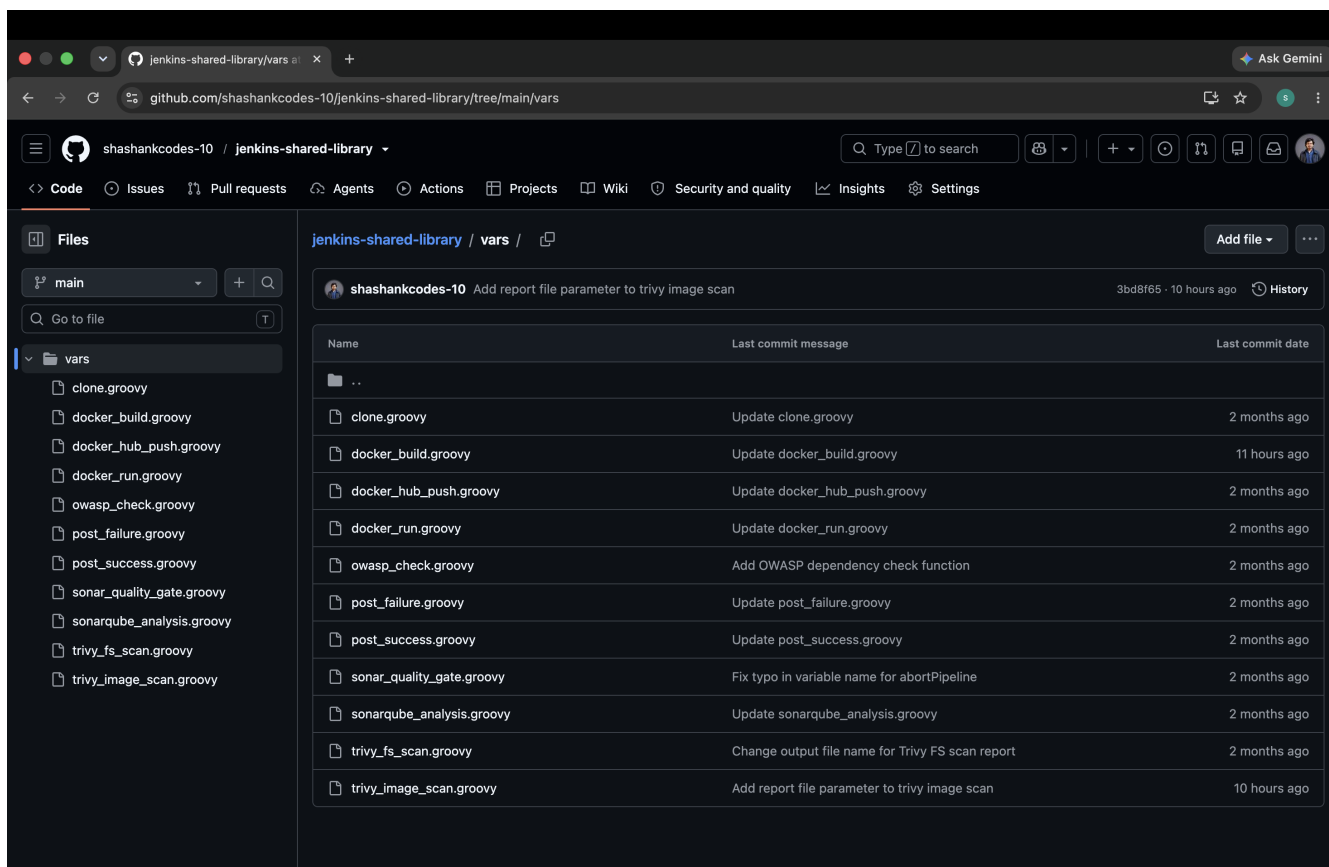


Figure 26 — Jenkins Shared Library vars directory containing reusable pipeline steps.

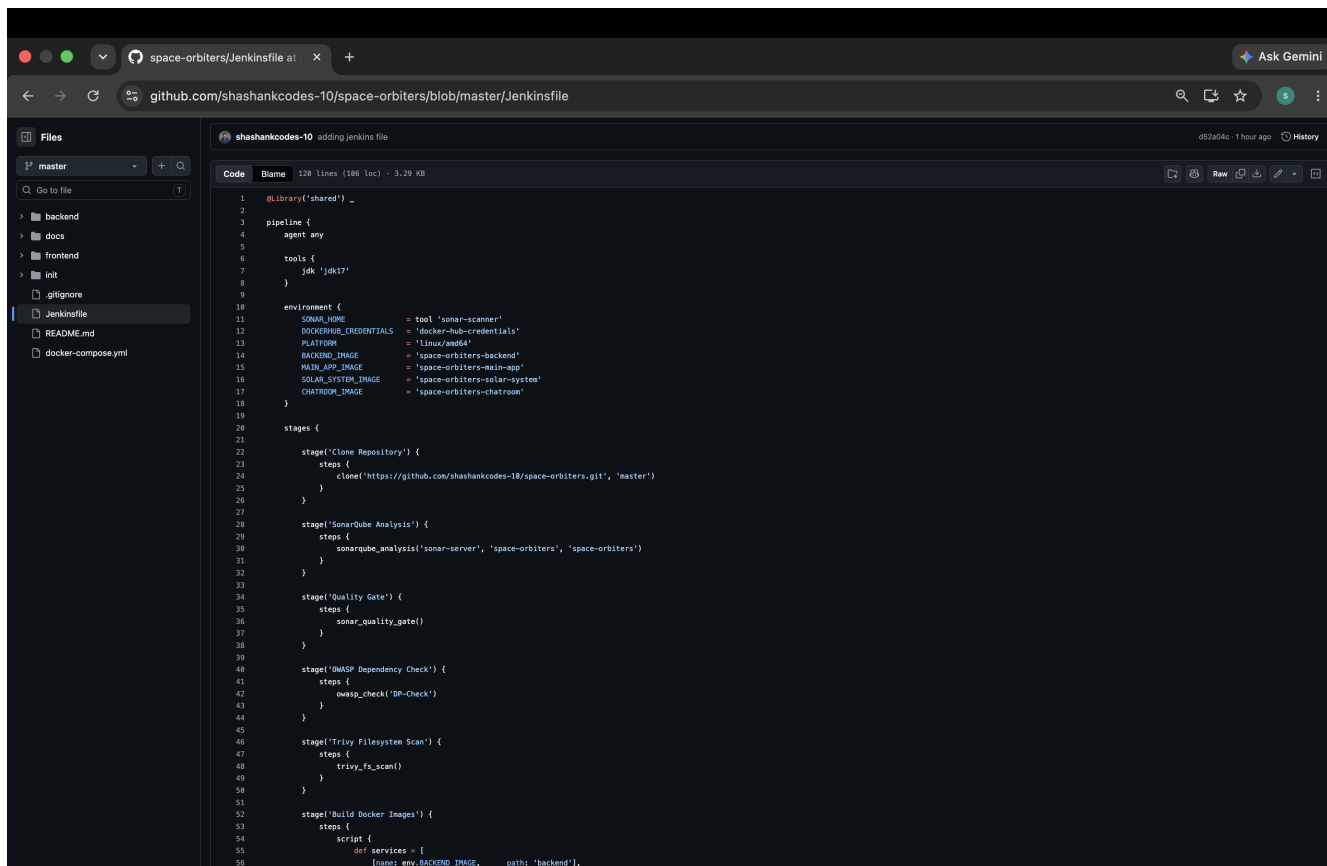


Figure 27 — Jenkinsfile for the Space Orbiters CI/CD pipeline.

Implementation Summary

The project was transformed into a containerized 3-tier application with PostgreSQL persistence and an AWS EC2 deployment. The implementation includes Docker and Docker Compose, multi-stage image optimization, Git/GitHub practices, and a structured service architecture.

Beyond the core project work, the deployment was extended with an Elastic IP, GoDaddy DNS, Nginx reverse proxy, Let's Encrypt HTTPS, and a Jenkins-based DevSecOps pipeline design.

Core project	Implemented and evidenced through the main steps of this report.
Persistent data	Evidence supplied through the linked Google Drive persistence video in Step 6.
Production enhancements	DNS, Elastic IP, Nginx, HTTPS/SSL, Certbot, and Jenkins are explicitly separated from the core requirements.

End of Report